

LocalHouseLLM Research

Independent AI Systems & Local Inference Research Division

KV Cache Optimization for Large Language Model Inference

Report 04 · KV Cache Deep-Dive Handbook | Prepared by LocalHouseLLM Research | Confidential Technical Compendium

KV Cache Optimization for Large Language Model Inference

LocalHouseLLM Research

Introduction

The key-value (KV) cache in a transformer stores the attention keys and values of previously generated tokens during autoregressive decoding. By reusing these cached states rather than recomputing them, systems trade memory for compute, dramatically improving throughput. However, for long context lengths or many parallel requests, the KV cache can become a gigantic memory bottleneck, spurring a surge of research on how to organize, compress, quantize, evict, prefetch, offload, and accelerate KV cache operations. This handbook surveys every known technique in exhaustive detail, intended for AI systems engineers and inference developers.

1. Historical Evolution

The KV cache idea originated with the advent of transformer-based LLMs. Early on, researchers noted that caching key/value tensors of past tokens eliminates recomputation and thus accelerates decoding. Initially, systems simply stored the cache in GPU DRAM and used CPU memory as a spillover. Over the 2020s, as LLMs grew and inference workloads demanded longer contexts, a parade of techniques emerged:

2022: Basic offload strategies (FlexGen-like systems) began exploiting PCIe and CPU RAM to hold KV cache beyond GPU DRAM. Early mixed-precision quantization (BF16, FP8) enabled modest KV compression.

2023: The first dedicated KV cache papers appeared. Kwon et al. introduced PagedAttention (SOSP'23), applying OS-style paging to attention to eliminate memory waste. NeurIPS'23 saw H2O (Heavy-Hitter Oracle), a greedy eviction policy with provable guarantees. Also in 2023 came Scissorhands (NeurIPS'23), which tracked token "importance" counts and pruned unimportant KVs, and SnapKV (arXiv 2024) showing 3-8x speed/memory gains by head-wise clustering of KV positions.

2024: Numerous specialized methods appeared. Ada-KV (NeurIPS'25) proposed head-wise adaptive budgets for eviction. Counter-Causal Surprise (2026) introduced a

novel eviction scoring by predicting past tokens from future context. Several KV compression schemes emerged: Palu (ICLR'25) used low-rank projection plus quantization to achieve $\sim 11\times$ KV compression; CSKV (NeurIPS workshop'24) applied a bi-branch low-rank channel reduction to cut 80–95% of KV size; PQCache (arXiv'24) used product quantization on keys; DecoQuant (arXiv'24) fused tensor decomposition with hybrid low-/high-bit quant for $\sim 75\%$ memory reduction. For prefetching/offload, 2024–25 saw SpeCache (arXiv'25) speculatively prefetching KV from CPU, and large-NVMe/CPU systems like PagedAttention/vLLM enabling KV sharing across requests. CXL memory began to be explored: Tang et al. (NeurIPS'24 workshop) used CXL-attached RAM for KV, cutting GPU requirements $\sim 87\%$.

2025–26: Quantization research intensified. Google's TurboQuant (ICLR'26) presented a near-optimal 2-stage vector quantizer (PolarQuant + 1-bit QJL) achieving “quality-neutral” KV quantization at ~ 3.5 bits. NVIDIA introduced hardware FP8 and even 4-bit (NVFP4) formats on next-gen GPUs. Learned policies appeared: Moschella et al. (2026) proposed KVP (RL-based KV Policy) to learn token-value importance. Advanced offloading systems like Samsung's LMCache used NVMe to serve hundreds of thousands of tokens, enabling 10–15x more concurrent users. The latest (2026) includes KVSculpt (ArXiv'26), a “distillation” approach optimizing a small continuous set of KVs, and DUAL-BLADE (ICDCS'26) which tightly couples NVMe and GPU DMA to trim 30–40% from latency.

This chronology underscores that KV cache optimization has moved from ad hoc offloading to a rich ecosystem of methods (paging, eviction, compression, quantization, prefetch, scheduling, hardware acceleration) all developed in rapid succession.

2. Complete Taxonomy

- **KV Memory Layout:** How cached key/value tensors are laid out in memory (GPU DRAM, CPU RAM, disk, CXL memory, etc). Techniques include contiguous allocation versus paged/segmented/hierarchical or ring-buffer layouts.
- **KV Compression:** Reducing KV cache size via lossy transformations — tensor decompositions (low-rank projections as in Palu), vector/product quantization (PQCache), and learned/semantic methods (KVSculpt, SnapKV).
- **KV Quantization:** Assigning lower precision to KV entries — FP16/BF16/FP8/INT8/INT4 and advanced schemes like TurboQuant, or per-channel/per-token schemes (Palu, Ada-KV).
- **KV Eviction/Pruning:** Selecting which cached KVs to drop when memory is full — H2O, TOVA, HashEvict, Counter-Causal Surprise, Scissorhands, SnapKV, Ada-KV, KVP, and predictive/statistical policies.
- **KV Prefetching:** Techniques to pre-load KV data before it's needed — SpeCache's speculative prefetch, hardware prefetch, and DMA/storage overlap (DUAL-BLADE).
- **KV Offloading:** Moving KV storage out of GPU memory — GPU→CPU, GPU→SSD/NVMe, GPU→CXL, and distributed/remote caches.
- **KV Scheduling:** How to schedule KV computations across tokens/requests — cache-aware routing (Ilm-d), token scheduling, pipeline prefill vs decode, asynchronous scheduling.
- **Kernel-Level Optimization:** Low-level GPU/CUDA methods for KV read/write — FlashAttention fusion, warp specialization, shared memory/register tiling, persistent kernels.
- **Compiler-Level Optimization:** MLIR/LLVM passes, IREE/TVM scheduling for

attention with cached KV, TorchInductor support, AITemplate kernels, graph rewrites.

- **Distributed KV Cache:** Techniques for multi-GPU/node systems — expert caches, hierarchical caching, cache synchronization, replication, and partitioning.

3. Deep Technical Explanations

KV Memory Layout

Contiguous vs Paged Layouts: The simplest layout is a growing contiguous buffer, which fragments with many varying-length requests. PagedAttention instead divides attention into fixed-size “pages” (e.g. one page per token), using free-list and paging to virtually map only needed portions — akin to OS virtual memory — achieving near-zero wasted space and allowing requests to share physical pages when they have identical prefixes.

Segmented / Hierarchical Allocation: Some systems segment “hot” (recent/pinned) and “cold” KV storage, moving seldom-used tokens into larger but slower tiers, implemented via a ring buffer plus a pool for O(1) append/drop.

Circular Buffers / Ring Buffers: In a single-request context, one can overwrite the oldest KV slots in round-robin fashion if capacity is fixed — a sliding window / FIFO recency eviction.

Memory Pools and Slab Allocators: Reduce fragmentation by allocating fixed-size blocks from a pool; slab allocators group blocks by size if KV blocks vary in size.

Compressed or Sparse Layouts: Some layouts compress groups of KV in memory (interleaved heads, sparse bitmaps, run-length encoding on K/V if sparsity is high).

KV Compression

Low-Rank / Factorization: Palu represents keys and values by projecting them into a lower-dimensional subspace via an inserted low-rank decomposition (SVD/Tucker) into each K/V linear layer, caching the smaller latent keys/values and reconstructing the full dimension on-the-fly. Palu achieved up to 91.25% KV-cache memory reduction (11.4x smaller) with negligible perplexity increase. CSKV uses low-rank shrinkage on the channel dimension with a “bi-branch” design (full-precision sliding window + compressed cache for older tokens), reporting ~80% KV memory reduction (up to 95% combined with quantization).

Token Merging / Clustering: KVSculpt creates a smaller set of synthetic “sculpted” KV pairs approximating the original attention behavior via alternating L-BFGS optimization and least-squares. Reduced attention KL divergence by 3.5–4x vs a simple select-and-fit baseline; requires a costly optimization pass per sequence.

Sparse / Entropy Methods: True sparsity is uncommon in dense transformers; some works apply residual quantization or learned autoencoders, though no prominent KV cache paper uses Huffman/arithmetic coding specifically.

Summary table of methods: Palu (2024, >90% comp.), CSKV (2024, ~80–95% comp.), SnapKV (2024, ~8.2x memory efficiency), Scissorhands (2023, 2–5x reduction with no quality drop), PQCache (2024, +4.6% quality gain vs baselines), DecoQuant (2024, ~75% reduction), TurboQuant (2025, near-lossless at ~3.5 bits), KVSculpt (2026, continuous distillation), Ada-KV (2024, head-wise adaptive

budgeting).

KV Quantization

- **FP8 (8-bit floats):** NVIDIA H100/A100 GPUs provide FP8/NVFP8; halves KV memory vs FP16/BF16.
- **Mixed Precision:** Keys and values stored in different precisions, or per-layer precision (Ada-KV implicitly does this).
- **Sub-8-bit:** NVIDIA's Blackwell GPUs support NVFP4 (4-bit floating), 75% smaller than BF16 and 50% smaller than FP8. LMCache combines FP8/FP4 to enable dozens of 128K-context users per GPU.
- **Extreme Quantization:** TurboQuant proposes random rotation followed by per-coordinate quantization and a 1-bit JL transform correction, achieving near-optimal distortion — quality-neutral at 3.5 bits per channel (~9x compression), slight degradation at 2.5 bits (~12x).
- **Asymmetric / Dynamic Quant:** DecoQuant adaptively quantizes outlier vs bulk tensor parts differently.
- **Zero-bit Compression:** TurboQuant's Quantized Johnson-Lindenstrauss (QJL) yields "zero-memory-overhead" one-bit summaries.
- **Training-Free vs Learned:** DecoQuant is data-free via tensor decomposition; most KV quant work aims to be calibration-free.

KV Eviction Policies

- **Recency (Window):** Evict oldest tokens / maintain sliding window; simple, no quality guarantee.
- **Attention-based:** Mean Attention drops lowest average-attention tokens; H2O uses a greedy submodular approximation with a near-optimality guarantee. HashEvict uses LSH to approximate lowest-attention tokens pre-attention, compressing 30–70% with minimal loss. Counter-Causal Surprise scores tokens by predictability from future context, removing highly-predictable (low-surprise) tokens.
- **Heavy-Hitter / Pivotal:** Scissorhands counts how often each token has been attended ("pivotality"), dropping persistently low-importance tokens (achieved 2–5x reduction with no quality drop).
- **Learned/Predicted:** Ada-KV computes theoretical bounds on attention loss per head to allocate cache budgets. KVP frames eviction as per-head reinforcement learning, significantly outperforming H2O/TOVA in experiments.
- **Predictive/Statistical:** Bayesian predictors estimate future reuse of KV blocks, evicting low-predicted-reuse blocks, outperforming simple LRU in simulations.
- **Hybrid schemes:** Many papers combine eviction with compression (e.g. Scissorhands + 4-bit quant).

KV Prefetching

Speculative Cache (SpeCache): Keeps a reduced-precision (e.g. INT8) copy of the entire KV cache on GPU; at each decode step, computes in reduced precision which tokens the new query will likely attend to, and speculatively fetches full-precision KVs from CPU ahead of time, overlapping via asynchronous streams. Shows near-zero information loss even with 10x less GPU KV memory.

CPU/GPU Hardware Prefetch: `cudaMemcpyAsync` for certain KV ranges when GPU is idle, `cuMemPrefetchAsync` for unified memory, or NVLink hardware caching. FlashAttention-4 kernels may proactively prefetch future K/V rows.

DMA/Storage Overlap: DUAL-BLADE implements double buffering — one thread issues PCIe copies of KV blocks from disk while others compute — showing up to 42% latency reduction.

KV Offloading

- **GPU→CPU (DRAM):** The simplest offload; systems like vLLM and DeepSpeed Infer spill KV to host memory. Samsung’s study shows offloading allows 1.7x more concurrent users with more consistent latency.
- **GPU→SSD/NVMe:** For extremely long contexts, NVMe stores “cold” KV blocks. LMCache demonstrates a CPU→NVMe tier yielding ~1.5x higher throughput at high concurrency and reducing 128K-context prefill time from ~11s to ~1.5s on cache hit; can sustain ~10x more simultaneous users at 128K context.
- **GPU→CXL (Memory Disaggregation):** Tang et al. show CXL-attached memory can reduce GPU memory needs by ~87% and yield 7.5x higher GPU utilization during prefill.
- **Network/Remote Caches:** Expert caches or shared KV via RDMA/remote CXL are conceptually possible but sparse in production.
- **Zero-copy & Unified Addressing:** cudaMallocManaged buffers let hardware migrate pages between CPU and GPU transparently; NVLink and GPUDirect RDMA enable zero-copy peer access.

KV Scheduling

- **Cache-Aware Routing (Distributed):** Systems like llm-d use an indexer tracking which node has which KV blocks, routing prompts to nodes with maximal KV overlap.
- **Token Scheduling:** Reordering attention computations across parallel streams to increase memory locality.
- **Pipeline Prefill vs Decode:** Overlapping prefill of new requests with decoding of ongoing ones.
- **Asynchronous Streams:** Using CUDA streams for prefetching and compute concurrently.
- **Overlap with Other Kernels:** FlashAttention-4 on SM100 implicitly schedules memory accesses via internal buffering.

Kernel-Level Optimization

FlashAttention/FlashInfer fusion reorders memory access to reduce KV read overhead; persistent/streaming kernels keep entire KV caches on-chip across iterations; warp specialization dedicates certain warps to KV caching operations; register tiling holds small-head-dimension KV vectors in registers; L2/cache optimization keeps soon-to-be-reused KV blocks cached; custom PTX/ASM tunes thread block sizes for KV gather/scatter throughput.

Compiler-Level Optimization

Automatic scheduling (TVM/TensorIR fusing “KV update” patterns), graph rewrites (replacing decoder layers with stateful incremental-decoder kernels), kernel generation (FlashAttention-4-style specialized code generators), automatic precision lowering (AITemplate/TensorRT mixed-precision calibration), and MLIR passes for “caching matmul” patterns (mostly proposals, limited public work).

Distributed KV Cache

Multi-GPU tensor parallelism shards K/V by head across GPUs; multi-node setups may use expert caches (parameter-server-style KV), cache synchronization protocols, hierarchical caching (shared NVMe/NFS backends), and NUMA/zero-copy considerations. Systems like llm-d's Router conceptually support cross-node cache awareness.

4. Mathematical Foundations

At generation step t , attention output is computed from cached K,V of prior tokens: $A_t = \text{softmax}(Q_t K_{\{1:t-1\}}^T) V_{\{1:t-1\}}$. Without caching, compute is $O(t^2d)$; with caching, per-step cost is $O(td)$ memory plus $O(td^2)$ compute for QK^T . H2O frames eviction as submodular maximization (greedy algorithm with $1-1/e$ approximation guarantee). Scissorhands and related methods rely on a “persistence of importance” hypothesis about cumulative attention weight. TurboQuant provides formal distortion bounds, coming within $\sim 2.7\times$ of the optimal bound. Complexity comparisons: naive attention $O(t^2d)$; after eviction to size k , $O(kd)$ per token; low-rank adds $O(rd)$ reconstruction; HashEvict is $O(nd)$ per token; Ada-KV/TOVA are $O(nh)$ per step. Full KV cache memory is $O(nd)$ floats; quantization reduces to $O(ndb/32)$; offloading moves memory out of GPU but total system memory remains $O(nd)$.

5. Implementation Details

Many proposals come with Triton kernels or custom CUDA (e.g. Palu's fused kernels for low-rank decode). Production/open-source tools: vLLM/LMCache (CPU/NVMe offload, prefix caching), FasterTransformer (FP8 KV, persistent cache), TensorRT (dynamic KV tensor sizing via ONNX), HuggingFace Transformers, llama.cpp (4-bit KV quant, no dynamic offload). Coding strategies vary from monkey-patching HuggingFace models (SnapKV) to requiring offline adapter training (KVSculpt, KVP).

6. Kernel-Level Considerations

CUDA resource limits (register file constraints for persistent kernels), shared memory vs L2 tradeoffs, bandwidth/stride alignment concerns for ring-buffer wraparound, PTX-level cache tagging, concurrency via double-buffered async copy (as in SpeCache), and FlashAttention-4's 128x128 tiling to reduce KV footprint on SM100.

7. Compiler-Level Considerations

XLA/MLIR passes for KV-specific fusion remain underexploited; PyTorch's lazy tensors could identify repeated cache-update subgraphs; TVM's `cache_read/write` primitives could allocate KV to faster memory; TensorRT's INT8 calibration is a natural fit for automatic KV precision reduction. Fully automatic KV cache insertion by compilers remains an open challenge.

8. Production Implementations

- **vLLM:** Multi-request batching with sharing, CPU/NVMe offload via LMCache, PagedAttention-like memory pool.
- **FasterTransformer:** FP8, fused attention, persistent cache output.
- **DeepSpeed:** CPU offload and empty-cache support, extensible.
- **TensorRT:** Dynamic sequence-length KV allocation, no built-in eviction.
- **OnnxRuntime:** Dynamic attention, no built-in compression.

- **LMCache (ByteDance/Samsung):** Targets KV offload to SSD.
- **llm-d:** Scheduler with an Indexer component tracking KV block location.
- **Academic repos:** H2O, SnapKV, Palu, Ada-KV all provide PyTorch code.

9. Hidden and Experimental Methods

Appendix ideas, GitHub issues, pull requests, workshop papers (CSKV and others from a NeurIPS 2024 Efficient NLP workshop), and engineering blog posts (Samsung's offload benchmarks, LMCache blog) surface less-publicized techniques. Methods with fewer than ~50 citations or little adoption — KVSculpt, HashEvict, Counter-Causal — are highlighted as high-potential but possibly brittle.

10. Comparative Benchmarks

A comparison table across methods (Baseline, FP8 KV, NVFP4 KV, Palu, CSKV, PQCache, DecoQuant, Scissorhands, SnapKV, Ada-KV, H2O, HashEvict, TOVA, Counter-Causal, KVP, SpeCache, DUAL-BLADE) evaluates: KV reduction, latency overhead, throughput impact, GPU utilization impact, VRAM savings, implementation complexity, and required kernel changes. Highlights: Palu achieves 91.25% (11.4x) reduction; NVFP4 gives ~75% reduction with negligible overhead; SpeCache enables up to 10x offload; DUAL-BLADE can offload 100% to NVMe with -33% latency and +2.2x SSD utilization.

Key comparisons: HashEvict vs H2O vs Counter-Causal (HashEvict is pre-attention/LSH-based and cheap per step; H2O is theoretically near-optimal but slower; Counter-Causal is foresight-based vs H2O's hindsight). Palu vs CSKV vs PQCache vs DecoQuant (each ~60-95% KV comp. via different mechanisms — low-rank, channel-shrink, product-quant, decomposition+quant — all fairly high integration effort). SpeCache vs KVSculpt vs TurboQuant (SpeCache's near-100% VRAM savings via full offload vs TurboQuant's simpler 75-90% memory cut via pure quantization).

11. Research Lineage

PagedAttention (Kwon et al., 2023) pioneered OS-style KV management, influencing vLLM's memory design. H2O (Zhang et al., 2023) formalized heavy-hitter eviction; TOVA (2024) and Scissorhands (2023) built on related persistence-of-importance concepts. HashEvict (2024) and Ada-KV (2024) cite H2O/TOVA as baselines. SnapKV (2024) and Scissorhands are contemporaries exploiting head-focus sparsity. Palu (2024) and CSKV (2024) arrived with similar low-rank ideas independently. PQCache (2024) comes from the embedding-retrieval community. DecoQuant (2024) and TurboQuant (2025) both address quantization via decomposition/transforms but from different angles. SpeCache (2025) is largely novel for spec prefetch. KVP (2026) integrates eviction ideas into a learned RL policy. KVSculpt (2026) and DUAL-BLADE (2026) represent the latest distillation and NVMe-direct offload lineages.

12. Future Directions

Ahead-of-their-time ideas (DUAL-BLADE, CXL offload) may become standard as hardware evolves. Underappreciated work includes SnapKV (huge practical gains, low citation count) and Counter-Causal Surprise (novel angle, brand new). Easy-to-reproduce methods include recency/low-rank approaches and SnapKV/Scissorhands. Likely industry standards: hybrid FP8/NVFP4 quantization combined with light eviction (Ada-KV) and multi-tier caches (GPU+SSD) as contexts exceed 1M tokens.

Complementary combinations (TurboQuant + Ada-KV + SpeCache) are plausible next steps. Research gaps include standardized cross-method benchmarks, holistic co-optimized algorithms, adaptive/online methods, deeper compiler integration, and error/hallucination analysis of compressed caches.

A next-generation inference engine would likely integrate: a hierarchical memory manager (PagedAttention/vLLM-style), multi-tier caching (GPU HBM + CPU DRAM + NVMe + CXL), FP8-by-default quantization with optional TurboQuant-level rotation, ensemble eviction policies (Ada-KV + H2O fallback), SpeCache-style speculative prefetch, OS-style shared allocation, FlashAttention-4-based kernels with fused quant/dequant, and a KV-index/directory for multi-node routing.

13. Comprehensive Bibliography

This report draws on peer-reviewed papers (NeurIPS, ICML, ICLR, workshops), arXiv preprints, systems whitepapers (SOSP, Samsung dev WP), engineering blogs, and code repositories, all cited inline throughout the text above.

LocalHouseLLM Research — Internal Technical Compendium

Document prepared by LocalHouseLLM Research. All findings, ratings, and citations herein reflect internal review of primary and secondary sources current as of the report date. For internal / partner distribution.